COL 1101

LECTURE-5

JULY 31, 2026

LAST CLASS: Time and Space complexity of Recursive algorithms

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

$T(n) = 2T\left(\frac{n}{2}\right) + cn$

$T(1) = c$

Finding time complexity

STEP 1: Write the recurrence relation

- ↳ recursive part
- ↳ non-recursive work
- ↳ base case.

STEP 2: Solve the recurrence

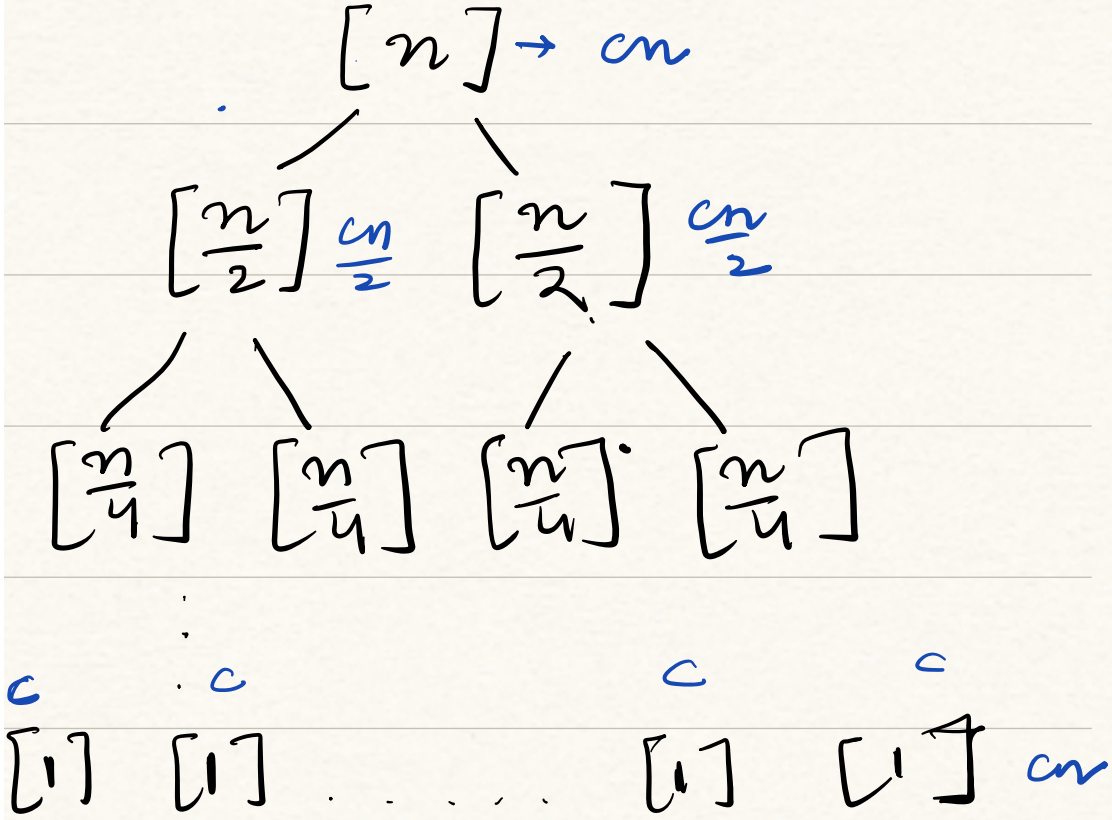
- ↳ Recurrence trace
- ↳ Unrolling

LAST CLASS: Merge sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

$O(n \log n)$

$T(n) \geq 2T\left(\frac{n}{2}\right) + cn$

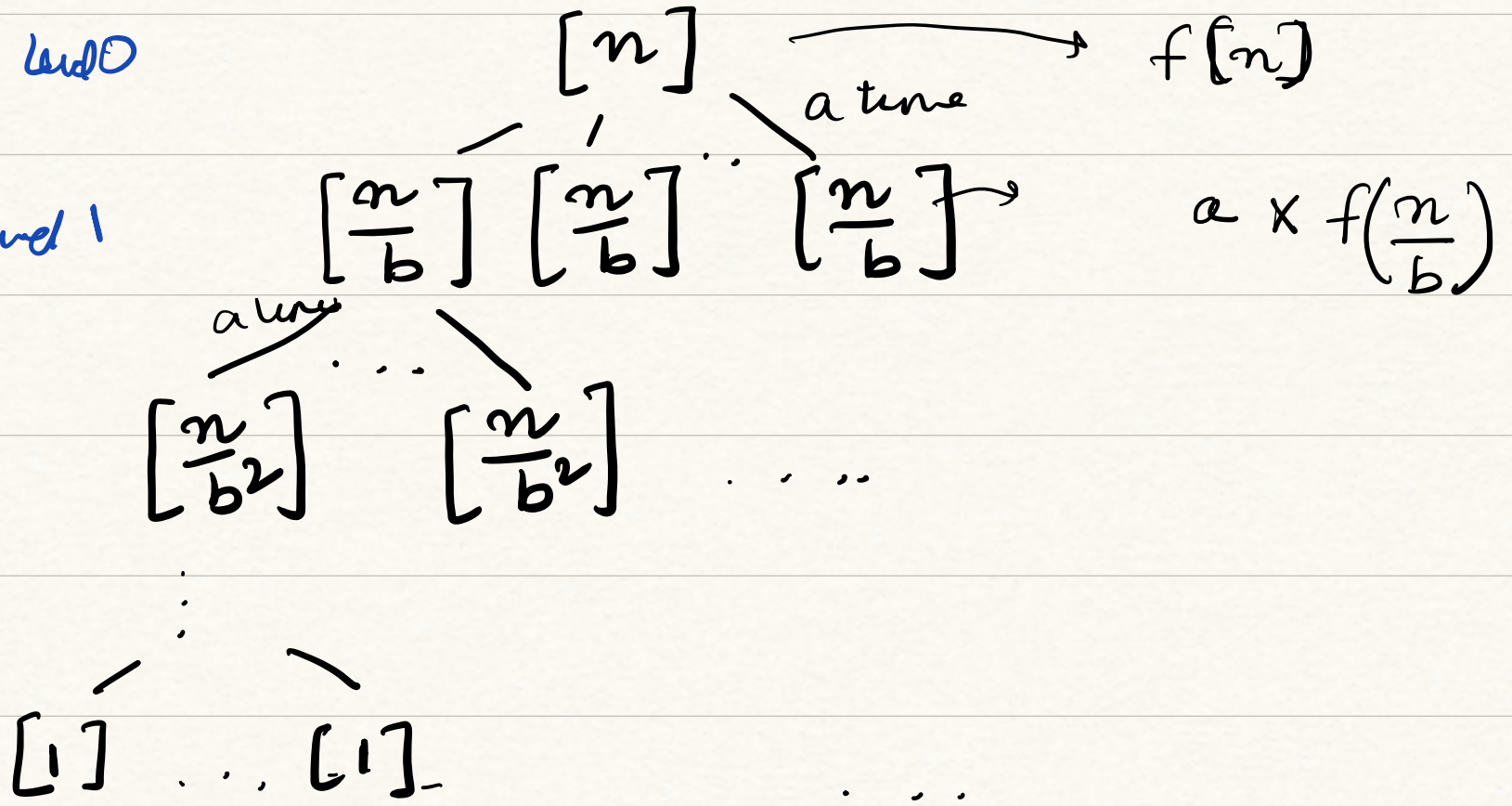


For the general form?

WHEN $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$

$a = 2$
$b = 2$
$f(n) = cn$
} Merge Sort

Recursion tree



How many nodes at each level ? a^i

How many levels ? $\log_b^n$

How many nodes at the last level ? $a^{\log_b^n} \rightarrow n^{\log_b^a}$

Work done at each level? $a^i f\left(\frac{n}{b^i}\right)$.

Assume $f(n) = kn^c$ where $c > 0$

$$\frac{ka^i n^c}{b^{ic}} = kn^c \left(\frac{a}{b^c}\right)^i \rightarrow \text{work done at each level}$$

either the first term is dominant or last

$r > 1$, $r < 1$ & $r =$

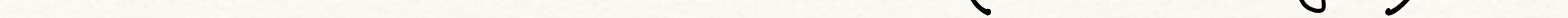
Work done at level 0 $= f(n)$

at last level $= cn^{\log_b a}$

Case 1: $f(n) > n^{\log_b a}$ then $T(n) = \Theta(f(n))$

Case 2: $f(n) < n^{\log_b a}$ then $T(n) = \Theta(n^{\log_b a})$

Case 3: $f(n) = n^{\log_b a}$ then $\Theta(n^{\log_b a} \log n)$



MASTER'S THEOREM

Formula for common shape

$$T(n) = a\,T\left(n/b\right) + f(n) \qquad a \geq 1,\ b > 1$$

Case 1: if $f(n)$ is $O\left(n^{\log_b a - \epsilon}\right)$ for some $\epsilon > 0$

$$T(n) = \Theta\left(n^{\log_b a}\right)$$

Case 2: if $f(n)$ is $\Theta\left(n^{\log_b a}\right)$

$$T(n) = \Theta\left(n^{\log_b a} \log n\right)$$

Case 3: if $f(n)$ is $\Omega\left(n^{\log_b a + \epsilon}\right)^{*}$ for some $\epsilon > 0$

$$T(n) = \Theta\left(f(n)\right)$$

* one more condition

MASTER'S THEOREM EXAMPLES

Merge sort : $T(n) = 2T\left(\frac{n}{2}\right) + cn$

$f(n) = cn \qquad a = 2 \;, \; b = 2$

$f(n) \qquad n^{\log_b a} \qquad n^{\log_2 2} = n$

$T(n) = \Theta(n \log n)$

Exercise : $T(n) = 8T\left(\frac{n}{2}\right) + n$

(Space can be reused)

SPACE COMPLEXITY : MERGE SORT

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

Goal: Space in addition to the input. Consider both:

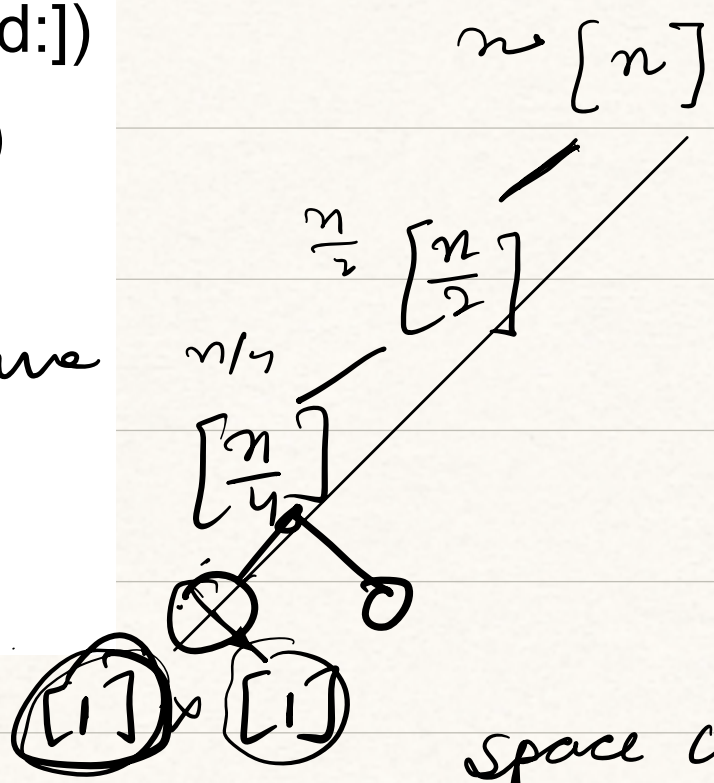
↳ non-recursive part

↳ recursion call stack

O(n)

↳ Non-recursive part

$O(\log_2 n)$



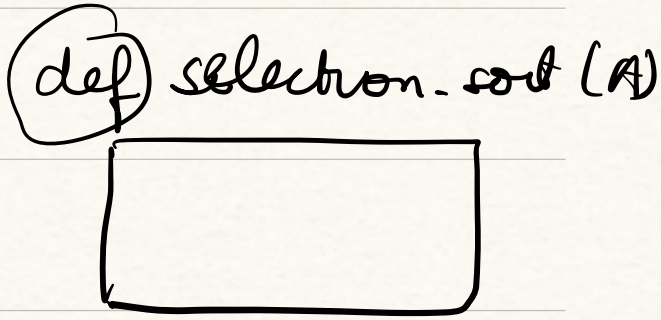
$O(n) + O(\log_2 n)$ is $O(n)$ → space complexity

LET'S TAKE A STEP BACK

Recall, a program is fundamentally manipulation of data.



PROGRAMS WE KNOW !



↳ A = [3, 2, 5, 1, 6]

sorted_A = selection_sort (A)

procedural programming

- data lives in a variable
- algorithm lives in a def
- function call joins the two

Is there anything wrong in keeping them apart ?

EXAMPLE: A SHOPPING CART

Goal: Implement a shopping cart in which items can be added or removed and it also stores the total

Example: A shopping cart

```
cart = {"items": [], "total": 0}
```

→ dictionary

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

```
add_item(cart, "Notebook", 80)
add_item(cart, "Pen", 20)
```

items : [(Notebook, 80), (Pen, 20)], total : 100

Invariant: sum of prices of items = total

Example: A shopping cart

```
cart = {"items": [], "total": 0}
```

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

add_item(cart, "Notebook", 80)
add_item(cart, "Pen", 20)

Invariant: total must always equal the sum of prices

WHAT CAN GO WRONG? PROBLEM #1

```
cart = {"items": [], "total": 0}
```

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

cart["items"].append(("Notebook", 80))

→ "cart" is not designed well

WHAT CAN GO WRONG? PROBLEM #1

```
cart = {"items": [], "total": 0}
```

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

cart["items"].append(("Notebook", 80))

Rule lived in functions but the data was reachable without them

One way to fix!

cart = {"items": []}

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

def cart_total(cart)

NEW FUNCTION!

One way to fix!

cart = {"items": []}

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

+ cart_total function

```
# Scattered across ten files, all assuming "there is a cart['total'] field":
show_badge(cart["total"])  # KeyError - the field is gone
if cart["total"] > 500: free_shipping()  # KeyError
```

One way to fix!

cart = {"items": []}

```
def add_item(cart, name, price):
    cart["items"].append((name, price))
    cart["total"] += price

def remove_item(cart, name, price):
    cart["items"].remove((name, price))
    cart["total"] -= price
```

+ cart_total function

```
# Scattered across ten files, all assuming "there is a cart['total'] field":
show_badge(cart["total"])  # KeyError - the field is gone
if cart["total"] > 500: free_shipping()  # KeyError
```

Problem #2: Representation leaks everywhere

What Is The Issue?

WHAT IS THE ISSUE?

The raw data is exposed.

How to fix this?

→ make data unreachable without functions

WHAT IS THE ISSUE?

The raw data is exposed.

How to fix this?

make cart's data reach only through sanctioned operations



MECHANISM: CLASS → Blueprint

```
class ShoppingCart:
    def __init__(self):
        self._items = []                    # the data, tucked inside

    def add_item(self, name, price):        # operations that belong to the cart
        self._items.append((name, price))

    def remove_item(self, name, price):
        self._items.remove((name, price))

    def total(self):                        # computed on demand — never stored
        return sum(price for name, price in self._items)
```

data attribute

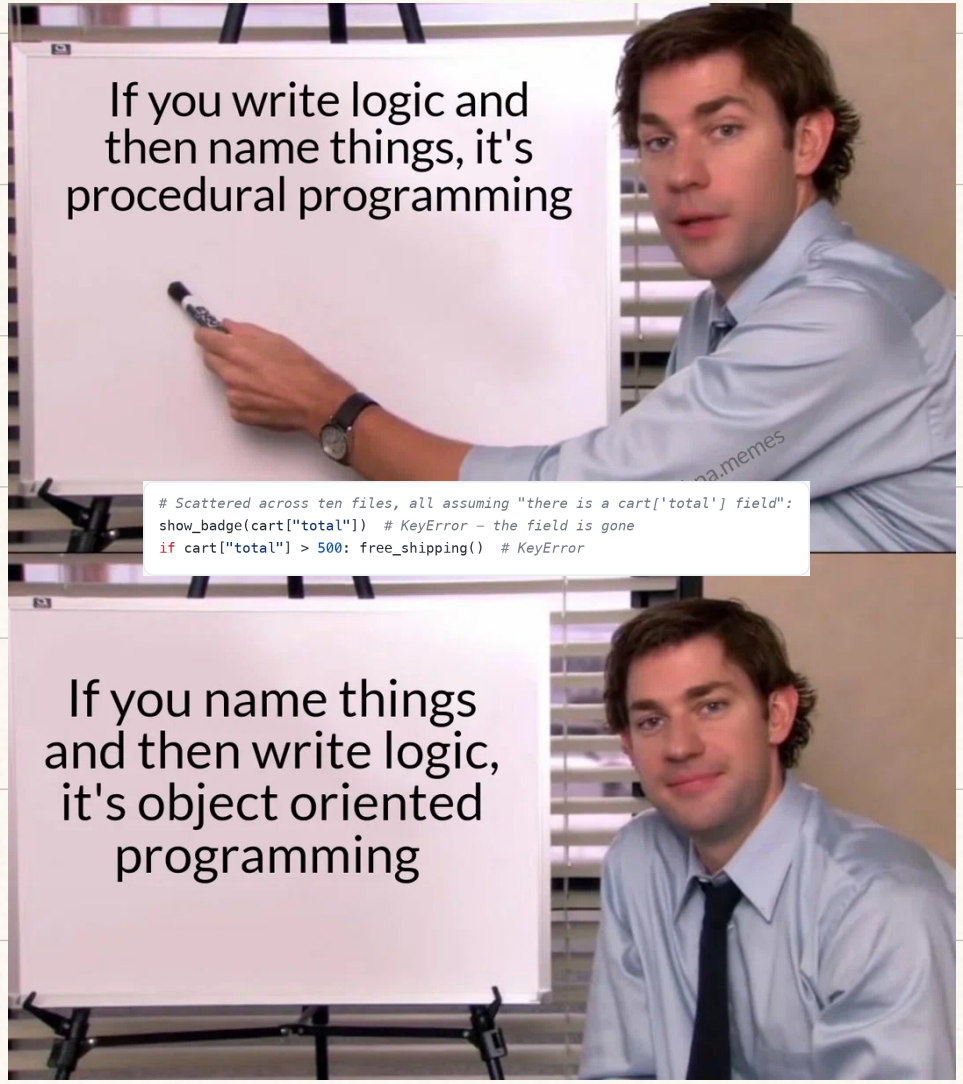
member elements

cart.lokesh = ShoppingCart()

cart.tanu = ShoppingCart()

Object: Instance of class

Procedural Programming vs Object-Oriented Programming



IDEA #1 : ENCAPSULATION

Bundling data with operations that act on it into a single unit.

→ Helps an object protect its own invariants

IDEA #2 : ABSTRACTION

Can change the internals without breaking things

```
class ShoppingCart:
    def __init__(self):
        self._items = {}                          # name -> [price, quantity]

    def add_item(self, name, price):
        if name in self._items:
            self._items[name][1] += 1             # seen before: bump the quantity
        else:
            self._items[name] = [price, 1]

    def total(self):
        return sum(price * qty for price, qty in self._items.values())
```

Separate what an object does [interface]

from how it does it

End Result: Modularity [Idea #3]

Because objects hide internals, a program consisting of objects is naturally modular.

OBJECTS ARE THE NATURAL LANGUAGE OF DATA STRUCTURES

data operation

stack : list , pop, push

queue : (list , queue . dequeue)